



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Lenguajes y Compiladores

Informe De Investigación

Análisis Sintáctico (Preguntas 1, 2 y 3)

Profesor:

Márquez Félix

Alumnos:

Aguilera Rhixeidys V-30.851.503

Albornoz Ronniel V- 24.889.173

Bravo Yvanna V- 28.530.717

Curbata Emilio V- 28.214.578

Ciudad Guayana, julio de 2026

Pregunta 1: Árbol de Sintaxis Abstracta (AST)

1.1 Definición

Un Árbol de Sintaxis Abstracta (AST, Abstract Syntax Tree) es una representación jerárquica en forma de árbol del código fuente de un programa, producida por el analizador sintáctico (parser) a partir de los tokens entregados por el analizador léxico. A diferencia del árbol de análisis o árbol de derivación (parse tree / concrete syntax tree), que refleja literalmente cada símbolo y cada producción aplicada de la gramática (incluyendo paréntesis, comas, punto y coma y símbolos no terminales intermedios), el AST abstrae esos detalles puramente sintácticos y conserva únicamente la estructura que tiene significado semántico para las fases posteriores del compilador (análisis semántico, optimización y generación de código) (Aho, Lam, Sethi & Ullman, *Compilers: Principles, Techniques, and Tools*, 2ª ed., Cap. 2 y 5).

1.2 Características

- **Jerárquico:** cada nodo representa una construcción del lenguaje (expresión, sentencia, declaración) y sus hijos representan sus componentes.
- **Abstracto:** omite tokens redundantes (paréntesis usados solo para agrupar, separadores, palabras reservadas que no aportan una operación). Por ejemplo, $(1 + 2)$ y $1 + 2$ producen el mismo nodo `BinOp('+')`.
- **Independiente de la gramática concreta:** distintas gramáticas (ambigua/no ambigua, LL/LR) que generan el mismo lenguaje pueden producir el mismo AST si se diseñan con ese objetivo (ver Pregunta 2).
- **Base para fases posteriores:** el AST es la entrada del analizador semántico (verificación de tipos, resolución de nombres) y del generador de código o intérprete.

- **Recorrible:** habitualmente se recorre con el patrón Visitor o de forma recursiva para evaluarlo, optimizarlo o traducirlo.

1.3 Dos ejemplos de entrada y su AST

Implementación: `pregunta1_ast/ast_ejemplo.py` (lexer con expresiones regulares + parser recursivo descendente LL que construye directamente los nodos del AST).

- **Ejemplo A: entrada:** $3 + 4 * 2 - 1$

```
BinOp('-')  
  
  BinOp('+')  
  
    Num(3.0)  
  
    BinOp('*')  
  
      Num(4.0)  
  
      Num(2.0)  
  
    Num(1.0)
```

Nótese que el AST refleja la precedencia de operadores (* antes que +/-) mediante la profundidad del árbol, sin necesidad de conservar paréntesis.

- **Ejemplo B: entrada:** $x = (1 + 2) * (3 - 4) / 5$

```
Assign('x')  
  
  BinOp('/')  
  
    BinOp('*')  
  
      BinOp('+')  
  
        Num(1.0)  
  
        Num(2.0)  
  
      BinOp('-')
```

Num(3.0)

Num(4.0)

Num(5.0)

Aquí los paréntesis de la entrada no aparecen en el AST: su único efecto fue alterar el orden de evaluación, y ese orden ya quedó representado por la posición de los nodos en el árbol (evaluado: $x = -0.6$).

Pregunta 2: Comparación entre análisis LL y LR

2.1 Definiciones

Criterio	LL(k)	LR(k)
Estrategia	Descendente (top-down), construye el árbol desde la raíz	Ascendente (bottom-up), construye el árbol desde las hojas
Derivación	Más a la izquierda (Leftmost)	Más a la derecha, en reversa (Rightmost in reverse)
Técnica típica	Recursiva descendente / tabla de predicción (FIRST/FOLLOW)	Pila + autómata de estados (ACTION/GOTO), shift-reduce
Recursión izquierda	No permitida, debe eliminarse reescribiendo la gramática	Permitida y natural ($E \rightarrow E + T$)
Poder expresivo	Subconjunto más restrictivo de las GLC	Reconoce una clase más amplia de gramáticas no ambiguas
Facilidad de implementación manual	Alta (se programa a mano fácilmente)	Baja (se requiere generador; construir tablas a mano es impráctico)

Herramientas típicas	Parsers recursivos descendentes, PEG/packrat (Python ≥ 3.9)	Bison/Yacc (LALR(1)), CUP, generadores LR(1)/SLR(1)
----------------------	---	---

2.2 Evidencia de uso actual

CPython reemplazó, a partir de la versión 3.9, su parser LL(1) por un parser PEG (recursivo descendente con retroceso/backtracking), en parte precisamente por las limitaciones de LL(1) frente a la recursión izquierda y a reglas ambiguas con un solo token de lookahead (PEP 617 — "New PEG parser for CPython", Python Software Foundation, 2020, <https://peps.python.org/pep-0617/>). Este es evidencia directa y reciente del uso (y de las limitaciones) de los parsers de tipo descendente en un lenguaje de producción masivo.

ANTLR (parser LL(*) adaptativo) es usado en producción por Twitter (parsing de más de 2 mil millones de consultas de búsqueda diarias), Hive y Pig (Hadoop), Oracle SQL Developer y sus herramientas de migración, NetBeans (parser de C++) e Hibernate (lenguaje HQL) — fuente: "About The ANTLR Parser Generator", [antlr.org](https://www.antlr.org/about.html), <https://www.antlr.org/about.html>.

Bison/Yacc (generadores LALR(1), un subconjunto eficiente de LR) siguen siendo el estándar de facto para compiladores de lenguajes de propósito general escritos en C/C++, documentado como referencia por el proyecto GNU (GNU Bison Manual, Free Software Foundation, <https://www.gnu.org/software/bison/manual/>).

En síntesis: los lenguajes con gramáticas simples, deterministas o donde se valora la facilidad de mantenimiento de un parser escrito/generado a mano tienden a usar LL/PEG (Python, muchos DSLs vía ANTLR); los lenguajes con gramáticas más

complejas o ambiguas y que priorizan el rendimiento de un autómata tabulado usan LR/LALR (C, C++, SQL vía Bison/Yacc).

2.3 Implementación de ambos análisis para el mismo lenguaje L

Se implementó el mismo lenguaje L (expresiones aritméticas con + - * / y paréntesis) con las dos estrategias, para comparar directamente su forma de operar sobre las mismas entradas:

- `pregunta2_ll_lr/ll_parser.py`: parser LL(1) recursivo descendente sobre la gramática sin recursión izquierda $E \rightarrow T E' ; E' \rightarrow + T E' \mid - T E' \mid \varepsilon$ (y análogo para T/T'). Imprime la traza de derivación, evidenciando la construcción de arriba hacia abajo.
- `pregunta2_ll_lr/lr_parser.py`: parser ascendente tipo shift-reduce dirigido por precedencia de operadores sobre la gramática natural con recursión izquierda $E \rightarrow E + T \mid E - T \mid T$. Mantiene una pila explícita y muestra en cada paso si la acción fue SHIFT o REDUCE, evidenciando la construcción de abajo hacia arriba.

Sobre las entradas de prueba $3 + 4 * 2 - 1$ y $(1 + 2) * (3 - 4) / 5$, ambos analizadores producen resultados numéricos idénticos (10.0 y -0.6 respectivamente) partiendo de gramáticas distintas para el mismo lenguaje L, lo cual ilustra que LL y LR son dos caminos distintos (descendente vs. ascendente) hacia el mismo AST semántico.

Una tabla LR(1)/SLR(1) completa con estados, ACTION y GOTO generados automáticamente es, en la práctica, el trabajo que realizan generadores como Bison/Yacc (ver Pregunta 3). Para esta demostración pedagógica de "cómo piensa" un parser ascendente se construyó a mano un parser shift-reduce por precedencia de operadores, técnica clásica y válida de análisis LR simplificado; esta simplificación se documenta explícitamente en el código fuente.

Pregunta 3: Generadores de analizadores sintácticos y manejo de errores

Generador	Tipo de parser	Entrada (gramática)	Manejo de errores
Flex	Generador léxico (regex/autómatas finitos), no un parser	Reglas regex → acción en C	Regla por defecto o reglas explícitas que reportan carácter inválido; la recuperación sintáctica la maneja el parser
Bison / Yacc	LALR(1) (ascendente)	GLC en notación BNF con acciones en C	Token especial 'error': descarta símbolos hasta sincronizar con una producción que contenga 'error' (modo pánico); yyerror() para el mensaje
ANTLR (v4)	LL(*) adaptativo (ALL(*)), recursivo descendente generado	Gramática EBNF combinada (léxico+sintaxis) en .g4	ErrorListener/ErrorStrategy: inserción/eliminación de un token, o sincronización tipo pánico con conjuntos FOLLOW
PLY (Python Lex-Yacc)	LALR(1) (como Bison, en Python)	Funciones Python con docstrings BNF	Función p_error(p) definida por el usuario; parser.errok() para recuperación

JavaCC	LL(k) recursivo descendente para Java	Gramática EBNF con producciones Java	Lanza ParseException/TokenMgrError con línea/columna y tokens esperados
--------	---	--	--

Fuentes Citadas

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson/Addison-Wesley.
- Van Rossum, G., Galindo Salgado, P., & Nikolaou, L. (2020). PEP 617 — New PEG parser for CPython. Python Software Foundation. <https://peps.python.org/pep-0617/>
- Parr, T. About The ANTLR Parser Generator. antlr.org. <https://www.antlr.org/about.html>
- Parr, T. (2013). The Definitive ANTLR 4 Reference. Pragmatic Bookshelf.
- Free Software Foundation. GNU Bison Manual — Error Recovery. <https://www.gnu.org/software/bison/manual/>
- Beazley, D. PLY (Python Lex-Yacc) Documentation. <https://ply.readthedocs.io/>
- JavaCC Project. JavaCC Documentation and Grammar Reference. <https://javacc.github.io/javacc/>
- Material del curso: Tema 5 (Análisis Sintáctico), Ing. Félix Márquez, UNEG, Lenguaje y Compiladores, trimestre 2026-I.